Perform authentication attacks using the right tools

Authentication is the gateway to every protected resource — so it's the most-attacked surface there is. The exam won't ask for definitions; it drops you into a scenario and asks **which attack, which tool, or what outcome**. The core mindset: **this is not just about cracking passwords**. The most powerful techniques never touch a plaintext password — they steal and replay hashes, tickets, and tokens, or break the protocol itself.

---

§00 the mental model: three layers of attack

Every 4.3 technique lives on one of three layers, defined by what it attacks. Placing an attack on its layer is half the battle — it tells you what the attacker needs and what defeats it. This is the frame to hang everything else on.

1 · the password
Dictionary · brute force · mask · spraying · credential stuffing — target the password itself. Differ by how much you know, online vs offline, and lockout evasion.

2 · bypass the password
Pass-the-Hash · Pass-the-Ticket · Pass-the-Token — never need the plaintext. Steal the authentication artifact (hash/ticket/token) and replay it.

3 · the protocol
Kerberos attacks · LDAP injection · SAML · OIDC — attack the framework itself, not one credential. Often the most powerful — they can affect an entire environment.

PICTURE IT A guarded building. **Layer 1** is guessing someone's PIN at the keypad. **Layer 2** is skipping the PIN entirely by using a photocopied keycard you lifted from their pocket. **Layer 3** is compromising the badge-printing machine in the back office — now you issue yourself any badge you like. Enterprise attackers live on layers 2 and 3, because copying a keycard beats guessing a strong PIN every time.

PART A Attack types

Thirteen techniques across the three layers — with the analogies and mnemonics that make the confusable ones stick.

2.1 MFA fatigue

Also called push bombing / prompt bombing. It doesn't break MFA technically — it breaks the human. The attacker already has valid credentials and floods the victim with push approvals until one is accepted.

It only works against **push-based** MFA (one-tap approve) — not TOTP codes or hardware tokens. Especially effective at night, or paired with a fake "IT support" call telling the user to approve. Defenses: number matching, showing login location/IP in the prompt, and capping MFA attempts.

PICTURE IT Ringing someone's doorbell over and over at 2 a.m. until they buzz you in just to make the noise stop. You already knew their address (the password) — you're just wearing them down on the second lock.

EXAM TELL Scenario: attacker **has credentials but is blocked by push notifications** → MFA fatigue. It's **user-targeted, not a cryptographic bypass**, requires **pre-obtained credentials**, and is **specific to push-based MFA**.

2.2 pass-the-hash (PtH)

Layer 2. NTLM authenticates with the password's hash, not the plaintext — so a stolen hash is the credential. Extract it and authenticate as that user without ever knowing the password.

Hashes come from **LSASS memory** (via Mimikatz), the **SAM** database (local accounts), or **NTDS.dit** (all domain accounts, on a DC). Reuse the hash against SMB, WMI, WinRM, RDP — any service accepting NTLM — to move laterally. Shared local-admin passwords across machines make this devastating. The hash never expires unless the password changes.

PICTURE IT You lift someone's keycard and run it through a photocopier. You never learn their PIN, but the copy opens every door their card did.

EXAM TELL Tied to **NTLM (not Kerberos)**; hashes from **LSASS via Mimikatz**; enables **lateral movement**; **never needs plaintext**. Kerberos-only environments resist classic PtH — but see Overpass-the-Hash (2.5).

2.3 pass-the-ticket (PtT)

Layer 2, Kerberos edition. Kerberos authenticates with tickets: you get a **TGT** from the KDC, use it to request **TGS** tickets for services. Steal a ticket from memory and inject it into your session to impersonate the victim.

Silver ticket

Forge a **TGS** for one specific service using that **service account's** NTLM hash. **No KDC contact** — pure offline forgery, so it's stealthier.

Golden ticket

Forge a **TGT** using the **krbtgt** hash → request any TGS for any service. **Domain-wide, persistent** access; survives until krbtgt is rotated **twice**.

THE DIFFERENCE **Silver = Single service** (service-account hash, no KDC). **Golden = the Grand key** (krbtgt hash, whole domain, near-permanent). A **Diamond** ticket modifies a real TGT instead of forging one, to evade detection.

PICTURE IT A festival wristband. PtT = reusing someone's already-stamped band. Silver = counterfeiting a band for one stage without asking the box office. Golden = stealing the wristband printer's master template (krbtgt) so you can print any band, for any stage, all weekend.

EXAM TELL PtT is **Kerberos-based** (vs PtH = NTLM). Tickets from **LSASS**; **Rubeus** is the tool. Golden = **krbtgt + domain-wide**; Silver = **service-account hash + one service + no KDC**. The exam distinguishes PtH vs PtT by the **protocol** named.

2.4 pass-the-token

Layer 2, modern edition. Web/cloud apps authenticate with tokens issued after login. Steal a token and replay it — no password or hash needed.

- **JWTs** — three Base64 parts (header.payload.signature); claims carry identity/roles. Weak/absent signature validation lets you modify the payload; a stolen token replays until it expires.
- **OAuth access tokens** — bearer tokens: whoever holds it can use it. Intercept = use, no extra credentials.

- **Session tokens** — in cookies/local storage; stolen via XSS, interception, or fixation → session hijack.

JWT-specific: **algorithm confusion** (swap RS256→HS256, sign with the public key), **none-algorithm** (set `alg: none`), **claim manipulation** (change roles when signatures aren't checked). Cloud: an SSRF against the AWS instance-metadata service (IMDS) can pull temporary credentials.

EXAM TELL Common in **web apps and cloud**; differs from PtH (NTLM) and PtT (Kerberos) — it targets **token-based auth**. **JWT manipulation** is the sub-technique; tokens stolen via **XSS or interception** get replayed. Tool: Burp Suite.

2.5 Kerberos attacks

Layer 3 — abusing the AD authentication protocol itself. The two "roasting" attacks extract crackable material; the ticket forgeries live under PtT; DCSync steals everything.

Kerberoasting

Any domain user requests a **TGS** for an account with an **SPN** (service account). The ticket is encrypted with the service account's hash → crack **offline**. Service accounts have weak, stale passwords. **Needs a low-priv domain account.**

AS-REP roasting

Targets accounts with **Kerberos pre-auth disabled**. Anyone can request an AS-REP, which contains data encrypted with the user's hash → crack offline. **Needs NO domain credentials.**

THE DIFFERENCE Both harvest hashes to crack offline. **Kerberoasting needs a domain account** and hits **SPN service accounts**; **AS-REP roasting needs nothing** and hits **pre-auth-disabled accounts**. Tools: Rubeus, Impacket (GetUserSPNs.py / GetNPUsers.py).

Also: **Overpass-the-Hash (pass-the-key)** converts an NTLM hash into a Kerberos TGT — bridging PtH and PtT so a hash works in Kerberos-only environments. **Skeleton Key** injects LSASS malware on a DC to add a master password (needs DA, dies on reboot). **DCSync** abuses AD replication — with replication rights (usually DA) you impersonate a DC and pull any account's hash, including **krbtgt**, without touching NTDS.dit (Mimikatz `lsadump::dcsync`).

MNEMONIC **AS-REP = Absolutely no creds.** Kerberoasting needs a Key (a domain account) to ask for the TGS. And **DCSync** = "sync like a Domain Controller" — you have to be trusted as a DC (replication/DA rights) to run it.

EXAM TELL Match the attack to the **access level**: no creds → AS-REP roasting; low-priv domain user → Kerberoasting; replication/DA → DCSync. "Crack service-account passwords" → Kerberoasting (Rubeus + Hashcat).

2.6 LDAP injection

Layer 3. Like SQL injection, but the target is a directory service query (Active Directory / OpenLDAP). Unsanitized input alters the LDAP filter's logic.

LDAP filters look like `(&(uid=USER)(password=PASS))`. Injecting special characters (`*`, `(`, `)`, `\`) can make the filter always-true — a `*` username may match any user; crafted input can bypass authentication entirely. Blind LDAP injection enumerates directory attributes (usernames, groups, emails) via true/false responses. Mitigation: escape special characters, parameterize where supported, least-privilege service accounts.

EXAM TELL **Conceptually parallel to SQL injection** (the exam uses this comparison) but hits **directory services**; it manipulates **filter logic** for **auth bypass or info disclosure**. Trigger: a web app that authenticates against Active Directory.

2.7–2.9 the password-guessing family: dictionary · brute force · mask

Layer 1. Three ways to produce password candidates, differing by how much the attacker assumes.

ATTACK	HOW IT GENERATES CANDIDATES	BEST WHEN
Dictionary (2.7)	Tests a **wordlist** of likely passwords (RockYou ≈ 14M). Rules transform words (capitalize, a→4, append digits/symbols).	Users pick common/predictable passwords
Brute force (2.8)	**Exhaustive** — every combination in a charset/length. No assumptions. Each extra character multiplies the space.	Short passwords or fast hashes (MD5, NTLM) with GPU
Mask (2.9)	A **template** per position — tests only combinations matching a known pattern. Exploits complexity-policy predictability.	The password policy is known

Mask charset placeholders: `?l` lowercase, `?u` uppercase, `?d` digit, `?s` special, `?a` all. Example `?u?l?l?l?l?d?s` catches Password1! and millions like it — because complexity rules push everyone to "capital first, digit + symbol last."

MNEMONIC Mask charset in order: **l-u-d-s-a** — "**l**ower, **u**pper, **d**igit, **s**pecial, **a**ll." And the key insight: **a mask attack is what you reach for the moment a question mentions the password-complexity policy** — the policy hands you the template.

EXAM TELL The seam: **dictionary = targeted list, brute force = exhaustive, mask = pattern-based**. "Complexity requirements known" → mask (Hashcat `-a 3`). Fast hash + GPU → brute force is feasible; bcrypt/Argon2 → not.

2.10–2.11 password spraying vs credential stuffing

Layer 1, but inverted — both spread across many accounts, so they dodge lockout. The exam constantly tests the difference.

Password spraying

One (guessed) password against **many accounts**, then the next password. Common picks: Welcome1, Summer2024!, Company123!. Never exceeds any single account's lockout threshold. Statistical: someone uses a weak password.

Credential stuffing

Known username:password pairs from breaches, tested for reuse. Doesn't guess or crack — the passwords are already valid somewhere. Exploits ~65% password reuse. Breaches: LinkedIn, Collection #1.

THE DIFFERENCE **Spray = Suspected** passwords you guess (one password, many users). **Stuff = Stolen** pairs you know (from a breach). Spraying's tool is **CME**; stuffing uses breach dumps + automation. MFA defeats both.

EXAM TELL "One common password across all accounts" → spraying (CME). "Breach-database credentials against a corporate portal" / "previously compromised credentials" → stuffing. Both **avoid lockout** by never hammering one account.

2.12–2.13 federation attacks: OIDC vs SAML

Layer 3 — attacking Single Sign-On. Both establish trust between an identity provider and an app; both fall to signature-validation failures and replay. The split the exam wants is the format.

OIDC (JSON/JWT)

Identity layer on OAuth 2.0 ("Sign in with Google"). The **ID Token** is a JWT. Attacks: **signature-validation failure** (modify claims, `alg:none`), **replay** when **nonce/expiration** aren't enforced, **redirect-URI** hijacking, WebFinger enumeration, SSRF via dynamic client registration.

SAML (XML)

Enterprise SSO. XML **assertions** signed by the IdP. Signature attack: **XML Signature Wrapping (XSW)** — duplicate the signed assertion and slip in a malicious one; the validator checks the real copy, the app processes the fake. Also token manipulation (change **NameID**), replay (NotBefore/NotOnOrAfter), signature stripping, IdP impersonation.

THE DIFFERENCE **OIDC = JSON/JWT, modern web**; **SAML = XML, enterprise SSO**. Both hinge on **signature validation** — break it and the trust collapses. The signature XSW attack is uniquely SAML. Tool for both: Burp Suite (Decoder for the Base64).

PICTURE IT The bouncer trusts a signed guest list faxed from HQ. **XSW** is stapling your name to the bottom of the page while HQ's signed original stays on top — the bouncer verifies the signature on page one and reads the name on page two.

EXAM TELL "Manipulating a SAML assertion / changing the username claim" → SAML token manipulation / **XSW**. "Modifying JWT claims because signature validation is absent" → pass-the-token / JWT manipulation (Burp). Remember: **SAML = XML, OIDC = JSON**.

PART B Tools

Eight tools — cracking (offline), spraying/guessing (online), AD mapping, and web-auth manipulation.

3.1 CrackMapExec (CME)

The "Swiss Army knife" of Windows AD post-exploitation — validate credentials, enumerate, and execute across many hosts at once. Protocols: SMB, WinRM, LDAP, MSSQL, RDP.

For 4.3 it's the tool for **password spraying** and **pass-the-hash** lateral movement (accepts hashes directly). It enumerates shares, sessions, logged-on users, and — critically — **where privileged users are logged in** (targets for credential theft). Output: `[+]` success, `(Pwn3d!)` = admin access.

EXAM TELL "Automated credential testing across a Windows domain," spraying, or PtH lateral movement → CME. **Post-exploitation** tool (after initial access). `(Pwn3d!)` = admin.

3.2 Responder

LLMNR/NBT-NS/MDNS poisoning — captures **Net-NTLMv2** hashes by answering Windows name-resolution broadcasts. The entry point of the credential-acquisition chain.

When DNS fails, Windows broadcasts "who is `fileserver`?"; Responder answers "me," and the victim authenticates to the attacker, yielding a crackable hash. Feeds directly into Hashcat/JtR (crack) or relay (see 4.2).

EXAM TELL "On an internal network **capturing Windows authentication broadcasts**" → Responder (LLMNR/NBT-NS poisoning). It captures **Net-NTLMv2** for offline cracking.

3.3–3.4 Hashcat vs John the Ripper — offline cracking

Both crack captured hashes offline (fast, no lockout, undetectable). The exam splits them on GPU-vs-CPU and auto-detect.

Hashcat

GPU-accelerated, world's fastest. You must specify the hash type (`-m`). Attack modes: `-a 0` dictionary, `-a 3` mask, `-a 6/7` hybrid. **Fast hashes** (NTLM `-m 1000`, MD5) fly; **bcrypt** crawls. Keeps a **potfile** of cracked hashes.

John the Ripper

Auto-detects the hash format — no need to specify. CPU-optimized, easiest to start. Modes: single, wordlist, incremental. The **Jumbo** edition adds hundreds of formats (shadow, SAM, Kerberos, PDF, ZIP).

THE DIFFERENCE **Hashcat = faster/GPU**, you name the hash type; **John = easier/auto-detect/CPU**, Jumbo = broad formats. Unknown hash type → John. Big GPU job → Hashcat. Both use **RockYou** + rules; both are the answer for the mask attack (Hashcat especially, `-a 3`).

MNEMONIC **Hashcat = Hardware (GPU) + you state the Hash type. John = Just run it** (auto-detects). If the scenario stresses "didn't know the hash format," that's John's tell.

EXAM TELL Offline + mask + GPU → Hashcat (`-a 3`). Auto-detect / CPU / unusual format → John (Jumbo). Both are **offline** — contrast with online tools below.

3.5 · 3.7 Hydra & Medusa — online cracking

Both perform online brute-force/dictionary/spraying against live services (SSH, FTP, HTTP forms, SMB, RDP, databases). Parallel via `-t`. Both carry lockout + detection risk.

Hydra has the broadest protocol support and is the more commonly referenced. **Medusa** is parallel + modular (`-M` selects the module, `-C` a combo file), known for stability in large tests. The exam may use either as the "online brute-force" answer — what matters is that both are **online**, unlike Hashcat/JtR.

EXAM TELL Live service, no captured hash → Hydra or Medusa. Both **online** (lockout risk). Distinguish from Hashcat/JtR (**offline**). For a web login form specifically, Burp Intruder is often the better answer.

3.6 BloodHound

Uses **graph theory** to map Active Directory relationships and reveal the shortest **attack path** from any compromised user to Domain Admin. It doesn't attack — it shows you where to.

SharpHound collects the data; **BloodHound** (Neo4j + Cypher queries) visualizes and analyzes it. It surfaces **Kerberoastable accounts, DCSync rights, unconstrained delegation**, and where privileged sessions live. BloodHound CE is the modern edition.

MNEMONIC **Sharp**Hound **collects** (it's the sharp tooth that bites the data); **Blood**Hound **analyzes** (it follows the trail). Data in → attack path out.

EXAM TELL "Find the **shortest path from a compromised user to Domain Admin**" / "visualize AD privilege-escalation paths" → BloodHound. SharpHound = collector, BloodHound = analyzer.

3.8 Burp Suite

The industry-standard web-app testing platform — an intercepting proxy between browser and server. For 4.3 it's the tool for anything touching web/token/SSO authentication.

BURP COMPONENT	USE IN AUTH ATTACKS
Proxy	Intercept/modify login requests, cookies, JWTs, SAML assertions
Intruder	Automate credential attacks (see modes below)
Repeater	Manually replay/modify requests — token validity, auth bypass
Decoder	Base64 encode/decode — **JWTs and SAML** are Base64
Sequencer	Analyze session-token randomness (predictability → hijack)

Intruder modes map cleanly onto the password-attack family: **Sniper** (one position, one list — single-field brute/enumeration), **Pitchfork** (parallel lists, paired — credential stuffing), **Cluster bomb** (all combinations — multi-field brute force).

MNEMONIC Intruder modes: **Sniper** = one shot at one spot; **Pitchfork** = parallel prongs moving together (paired lists → stuffing); **Cluster bomb** = everything hits everything (all combinations → brute force).

EXAM TELL Any **web-application authentication** scenario → Burp is almost always the answer. **Intruder** for automated credential attacks, **Decoder** for JWT/SAML, **Proxy** to intercept and modify.

SYNTHESIS How the pieces connect

The chain from initial access to domain compromise, and the decision that routes every password attack.

the credential-acquisition chain

initial access → Responder capture Net-NTLMv2 → Hashcat/John crack offline → CME validate + spray across domain → Mimikatz dump LSASS (hashes/tickets) → Pass-the-Hash / Pass-the-Ticket → BloodHound map path to Domain Admin

A scenario that captures hashes, cracks them, then moves laterally is describing this chain — the exam wants you to name each step's tool and technique.

online vs offline — the one question that routes everything

Have captured hashes? → OFFLINE

- **Fast hash** (NTLM, MD5) → Hashcat + GPU
- **Slow hash** (bcrypt) → Hashcat, but dictionary-only realistically
- **Unknown type** → John the Ripper (auto-detect)

No hashes? → ONLINE

- **Web app** → Hydra (HTTP) or Burp Intruder
- **Windows domain** → CME (SMB/WinRM)
- **Network service** (SSH/FTP) → Hydra or Medusa

The single discriminator is whether hashes are in hand. Offline = Hashcat/JtR; online = Hydra/Medusa/CME.

MAP correlation matrices

Windows authentication

YOU HAVE / WANT	ATTACK	TOOL
NTLM hash → lateral movement	**Pass-the-Hash**	CME, Mimikatz, Impacket
Kerberos ticket → service access	**Pass-the-Ticket**	Rubeus, Mimikatz
Persistent domain access	**Golden Ticket**	Mimikatz (needs krbtgt hash)
One service, no KDC contact	**Silver Ticket**	Mimikatz (service-account hash)
All domain hashes	**DCSync**	Mimikatz (replication rights)
Crack service-account passwords	**Kerberoasting**	Rubeus, Hashcat
Attack path to Domain Admin	**AD mapping**	BloodHound + SharpHound

modern authentication

AUTH TYPE	ATTACK	TOOL
Push-based MFA	**MFA fatigue**	Manual / social engineering
JWT tokens	**Pass-the-token, JWT manipulation**	Burp Suite
SAML SSO	**XML Signature Wrapping**	Burp Suite
OIDC / OAuth	**Token replay, redirect hijack**	Burp Suite
LDAP directory auth	**LDAP injection**	Burp Suite, manual

password-attack selection

ATTACKER KNOWLEDGE	BEST ATTACK	TOOL
No information	Brute force	Hashcat `-a 3`, Hydra
Common passwords likely	Dictionary	Hashcat `-a 0`, JtR
Password policy known	Mask	Hashcat `-a 3` + mask
Many accounts, avoid lockout	Password spraying	CME, Hydra
Breach data available	Credential stuffing	Hydra, custom scripts
Hash captured	Offline cracking	Hashcat, JtR

CRAM The seven distinctions the exam tests

These are the exact pairs CompTIA blurs. Lock in the one-line separation.

THIS	VS. THAT	THE SEAM
Pass-the-Hash	Pass-the-Ticket	**NTLM** hash vs **Kerberos** ticket — which protocol is named
Online (Hydra/Medusa/CME)	Offline (Hashcat/JtR)	Live service, lockout risk vs captured hash, fast & silent
Dictionary	Brute force / Mask	Targeted **list** vs **exhaustive** vs **pattern** (policy-driven)
Password spraying	Credential stuffing	**Guessed** common passwords vs **known breach** pairs
SAML	OIDC	**XML** / enterprise SSO vs **JSON/JWT** / modern web
Golden Ticket	Silver Ticket	**krbtgt**, domain-wide vs **service account**, one service (no KDC)
Kerberoasting	AS-REP roasting	Needs a **domain account** (SPN accounts) vs needs **no creds** (pre-auth off)

EXAM Scenario red-flags → answers

Phrases the exam plants, and the technique/tool they point to. If you can pattern-match these, most of 4.3 is muscle memory.

THE SCENARIO SAYS…	→ ANSWER
Valid credentials but blocked by push notifications	**MFA fatigue**
Extracted hashes from LSASS without cracking them	**Pass-the-Hash** (NTLM) / **Pass-the-Ticket** (Kerberos)
Domain-wide persistent access using the krbtgt hash	**Golden Ticket**
Specific service access without contacting the KDC	**Silver Ticket**
One common password across all domain accounts	**Password spraying** (CME)
Breach-database credentials against a corporate portal	**Credential stuffing**
Internal network, capturing Windows auth broadcasts	**Responder** (LLMNR/NBT-NS)
Shortest path from a compromised user to Domain Admin	**BloodHound**
Manipulating a SAML assertion to change the username claim	**SAML manipulation / XSW**
Modifying JWT claims because signature validation is absent	**Pass-the-token / JWT manipulation** (Burp)
Password policy requires upper + digit + special	**Mask attack** (Hashcat)
Requesting TGS tickets for service accounts to crack offline	**Kerberoasting** (Rubeus + Hashcat)
No domain credentials but wants to crack account hashes	**AS-REP roasting** (pre-auth disabled)

MINDSET four questions that decode any scenario

What do you have?

Nothing, credentials, hashes, tickets, or tokens — this alone rules most options in or out.

What's the target?

Windows AD, web app, cloud, or a network service — decides the tool family.

What's the goal?

Initial access, lateral movement, persistence, or data.

What's the constraint?

Lockout policy, MFA, encryption, network position — the constraint often is the answer.

The intersection of these four almost always points to the right attack-plus-tool. Authentication attacks aren't isolated — they're a connected workflow from initial access to domain compromise.

REFERENCE

Keyword quick reference

The term most likely to anchor a scenario, with its one association.

KEYWORD	ASSOCIATION
LSASS	Where Mimikatz pulls NTLM hashes & Kerberos tickets
Net-NTLMv2	Hash type Responder captures for cracking
krbtgt	The hash behind a Golden Ticket (domain-wide)
SPN	Service Principal Name — target of Kerberoasting
pre-authentication	If disabled → AS-REP roasting (no creds needed)
DCSync	Replication abuse → all domain hashes (needs DA)
Overpass-the-Hash	Convert NTLM hash → Kerberos TGT
Pwn3d!	CME output — admin access confirmed
-a 3 / -m 1000	Hashcat mask mode / NTLM hash type
Jumbo edition	John the Ripper's broad-format build
SharpHound	BloodHound's data collector
XSW	XML Signature Wrapping — the signature SAML attack
nonce	Anti-replay value in OIDC; absent → replay works
NameID	SAML user identifier — change it to impersonate
alg: none	JWT attack removing signature validation
Intruder (Sniper/Pitchfork/Cluster bomb)	Burp credential-attack modes
RockYou	The canonical ~14M wordlist
bcrypt / Argon2	Slow hashes that resist cracking

PenTest+ 4.3 · authentication attacks & tools

3 layers · 13 attacks · 8 tools · 7 distinctions